RA1 Report

Sergio Fabregat López

April 7, 2026

1 Direct Illumination - `direct integrator`

The `direct integrator` shoots rays from the camera to the scene. When those rays hit a surface, they perform only a single bounce. Outgoing radiance to the camera is computed using the surface material's properties and the radiance sampled by the bounce ray, all divided by the probability of generating a bounce ray in that specific direction. As such, the `direct integrator` exclusively performs **direct illumination**.

The direction the bounce ray takes is known as the **incoming direction** ω_i, because it's the direction where light comes from. The direction from the intersection point towards the camera is known as the **outgoing direction** ω_o.

1.1 Importance Sampling

The incoming direction can be computed in many ways. Generating a completely random direction would work, but we want our image to be good looking before the heat death of the universe. We can do something better. The theory tells us that if we generate samples following a probability distribution (PDF) that resembles the function we're trying to integrate, our integration method, Monte Carlo, will converge faster. This is called **importance sampling**.

The integral we're trying to approximate is the one in the rendering equation (Equation 1), which is the product of three factors. We could have the PDF we use to generate ω_i be proportional to any of those factors.

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\omega_i \in S} f(\omega_i, \omega_o) L_i(x, \omega_i)(n \cdot \omega_i) d\omega_i \tag{1}$$

- f is the BSDF, a function that models the optical properties of the material of the object the ray has hit. If we use these material properties to model the PDF we're going to use to generate the bounce ray we're importance sampling the material.
- L_i is the incoming radiance. That is to say, light from a light source that hits the surface point where the ray from the camera intersected. We can generate a random point in the scene's light source and shoot the bounce ray towards it. If it hits the light, we sample its radiance and use it in our outgoing radiance computations. If it hits something else, this point is in shadow and we mark incoming radiance as 0. That is importance sampling the light.

Figure 1 and Figure 2 show the results of importance sampling the material and the light, respectively. As we can see by the better defined specular reflection, importance sampling the material works better for glossier materials. On the other hand, importance sampling the light looks way less noisy on all the other diffuse materials of the scene. So, what do we do in scenes like these where we have both diffuse and glossy materials? What do we sample, light or material? We can do both.

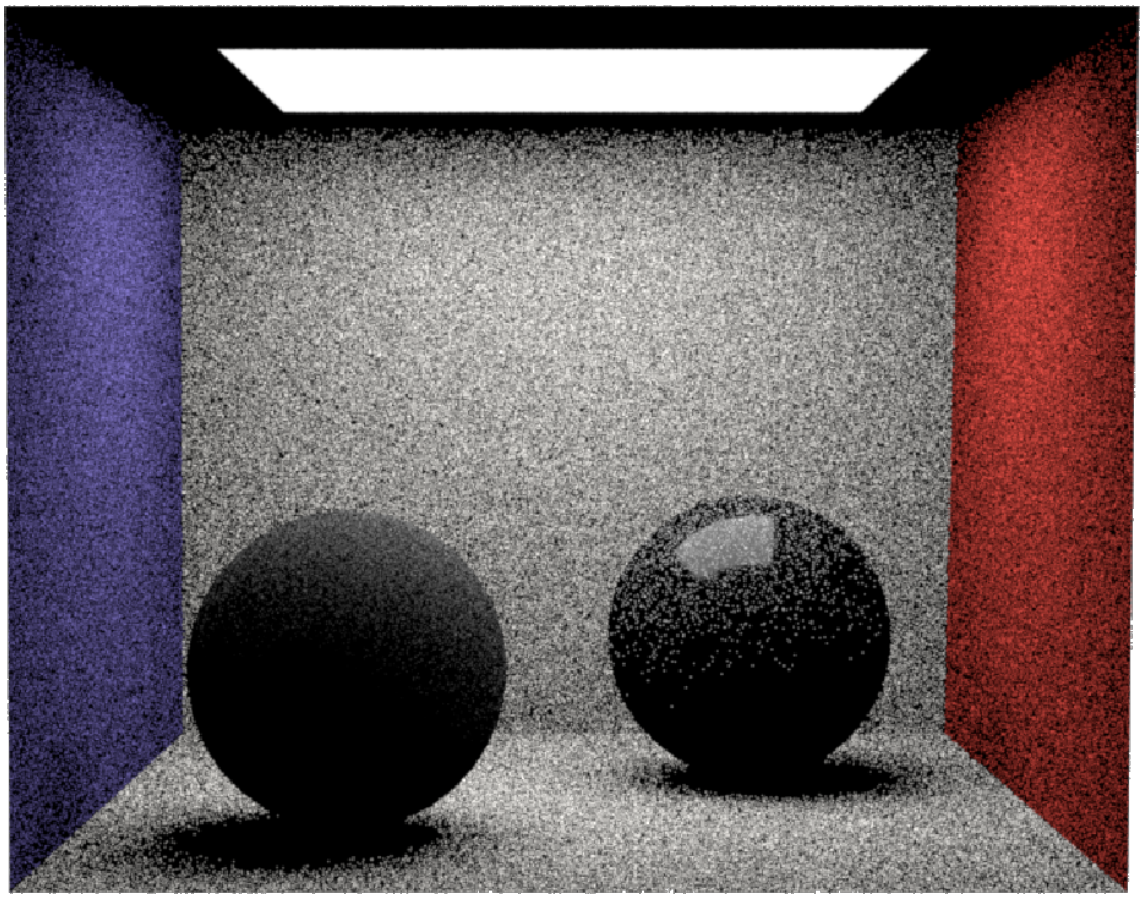

Figure 1: `direct integrator`, 16 Samples, Material Importance Sampling.

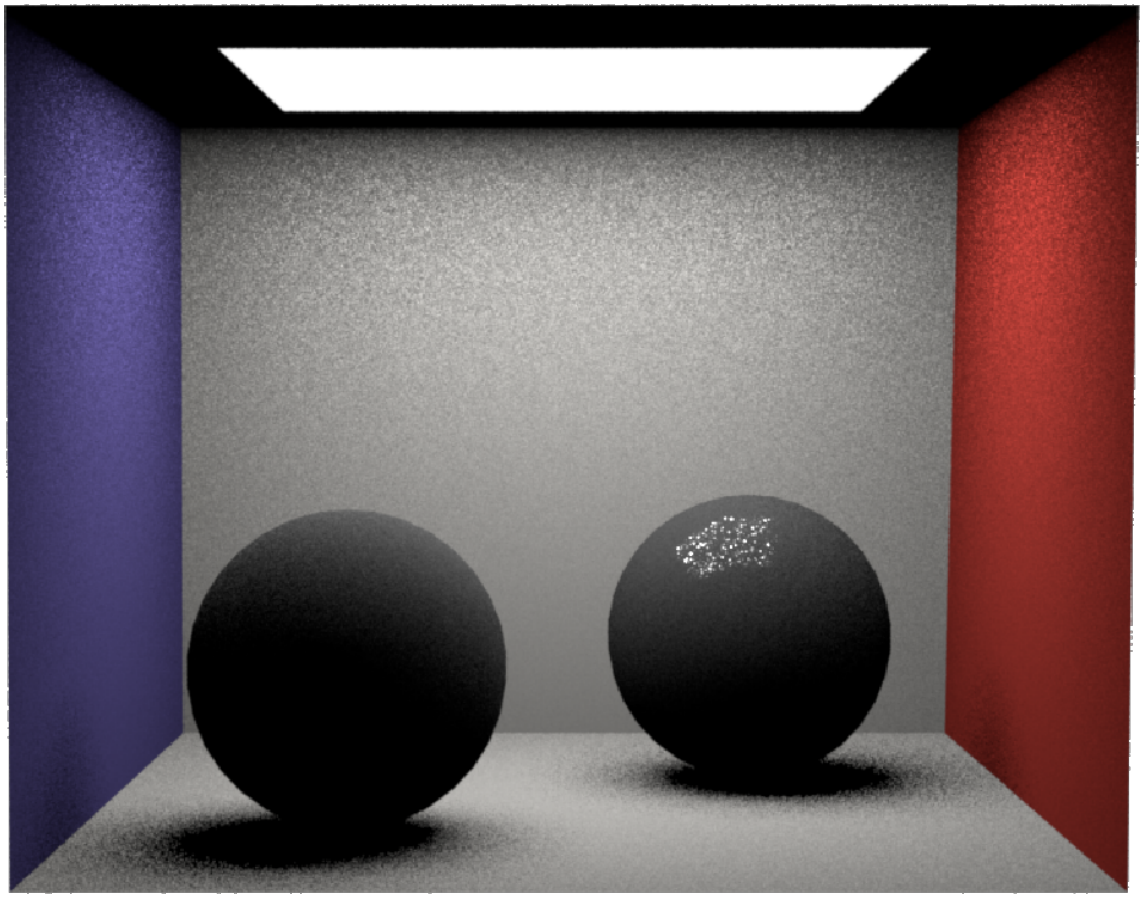

Figure 2: `direct integrator`, 16 Samples, Light Importance Sampling.

1.2 Multiple Importance Sampling

Multiple importance sampling (MIS) is a technique that combines the results of various importance sampling strategies into a single estimator.

Rather than simply averaging the results of material and light sampling, which would still retain the worst noise from both, MIS weights each sample. It does this by evaluating the probability of a sampled direction across both strategies. If a bounce direction is highly probable under the material PDF but improbable under the light PDF (such as a sharp specular reflection), MIS assigns a high weight to the material strategy and a near-zero weight to the light strategy.

As seen in Figure 3, combining these distributions through MIS yields the best of both worlds. It accurately handles the sharp highlights on the glossy sphere thanks to the material weight, while maintaining the smooth, low-noise diffuse areas and soft shadows provided by the light weight.

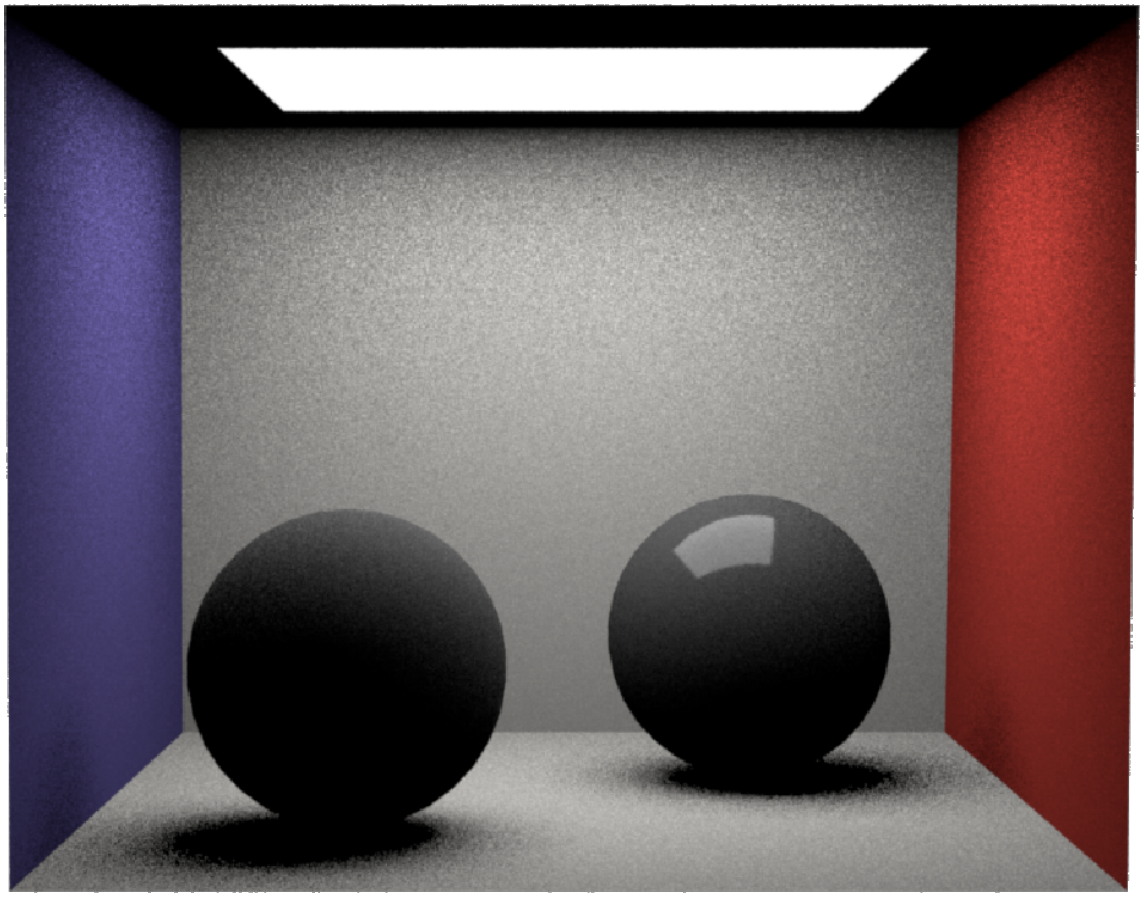

Figure 3: `direct integrator`, 16 Samples, Multiple Importance Sampling.

1.3 Radiance Dependent Probability

We've explained that sampling the light consists of choosing a point within the light, and shooting a ray towards said point. However, when there are multiple light emitters in the scene, we must choose which one to sample. We can choose one randomly with uniform probability, but, again, we can do better. Following importance sampling, we should give higher probability of being sampled to the emitters which influence the integral the most. And those are the ones that emit the most light. So, when building the probability distribution we give emitters a probability proportional to the total amount of power they emit.

It can be observed on Figure 4 that the render with uniform probability is noisier, especially around the bigger light. On the other hand, the radiance dependent probability render has more uniformly distributed noise and is overall less noisy.

As we've seen so far, direct illumination is very effective for visibility checks, so it's sufficient to get perfectly realistic soft shadows, but it fails at pretty much everything else. Mirrors, color bleeding, ambient occlusion, etc. All phenomena that can only be captured with **indirect illumination**.

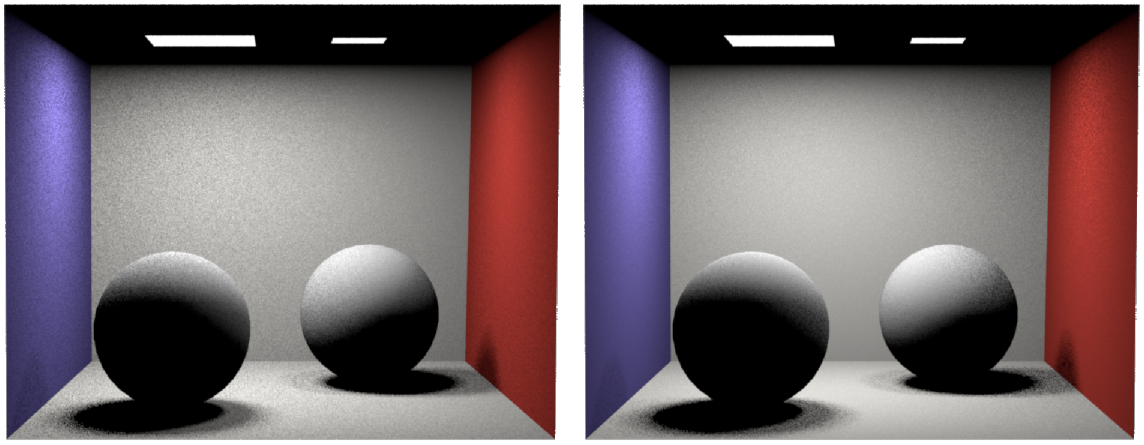

Figure 4: `direct integrator`, 16 Samples, Light Importance Sampling. On the left with uniform probability when choosing the emitter to sample, on the right with radiance dependent probability.

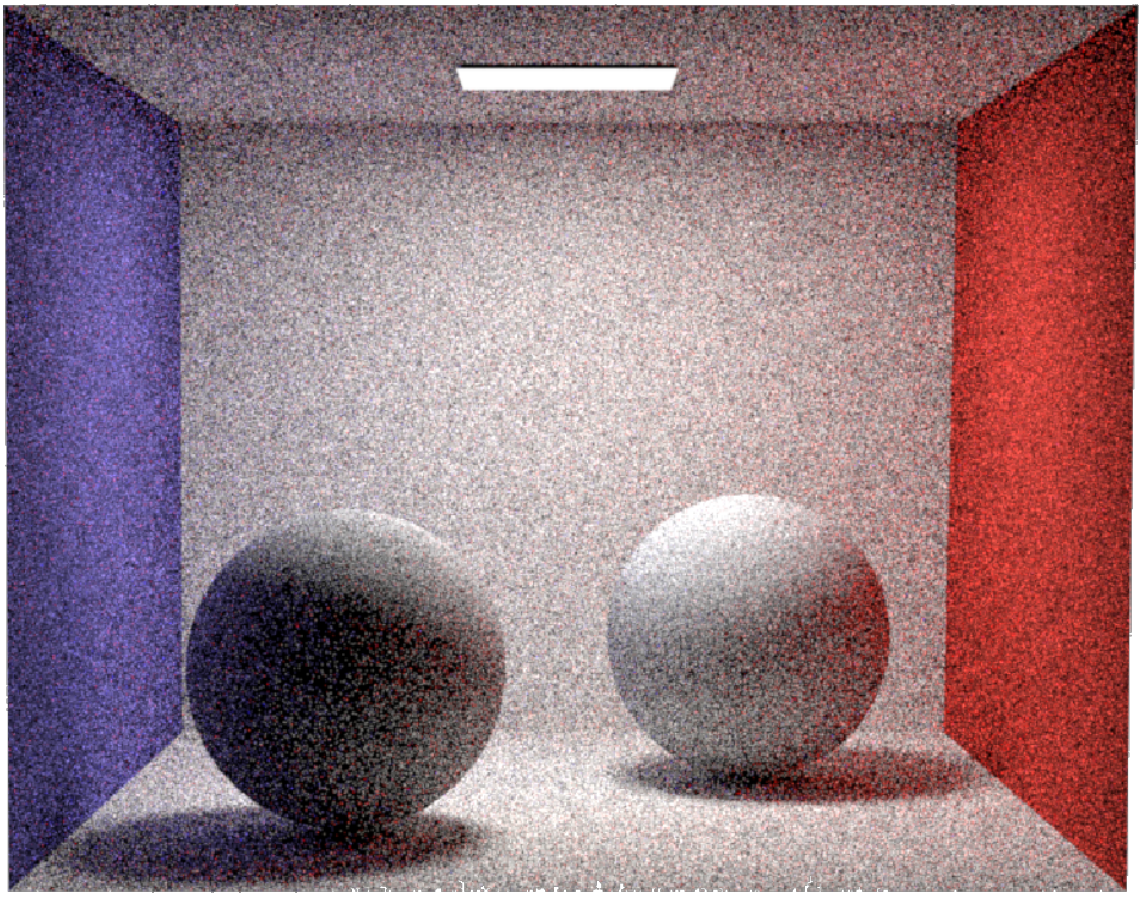

Figure 5: `pathtracer integrator`, 256 Samples, Diffuse materials.

2 Indirect Illumination - `pathtracer integrator`

The `pathtracer integrator` shoots rays from the camera into the scene. When those rays hit a surface, they bounce just as they do in the `direct integrator`. However, when that bounce ray hits a different point in the scene, it recursively bounces again. It will keep on bouncing throughout the scene until it reaches a light source (or until it's killed off via Russian Roulette). Once a light source is hit, the recursion terminates. As the recursive function calls return, the emitted radiance is propagated back along the path. At each intersection point, the radiance gathered from the subsequent bounce is treated as incoming light. This incoming radiance is then used together with the material's BSDF to compute the outgoing radiance directed toward the previous point. This way we're effectively tracing a path from the light to the camera like photons do in the real world, although we build the path from the camera to the light, in reverse order to how it happens in the natural world.

Using this path tracer, given enough samples, we would be able to render a perfectly realistic scene with almost all natural light phenomena present. However, as we can observe in the renders shown (Figure 5 and Figure 6), the images look very noisy even at relatively high sample counts. So far, at each ray-surface intersection point we're importance sampling the material and generating a single new bounce ray to follow in the recursion. But we can speed up the convergence process if we additionally sample the light at each intersection point. This is called next event estimation or NEE.

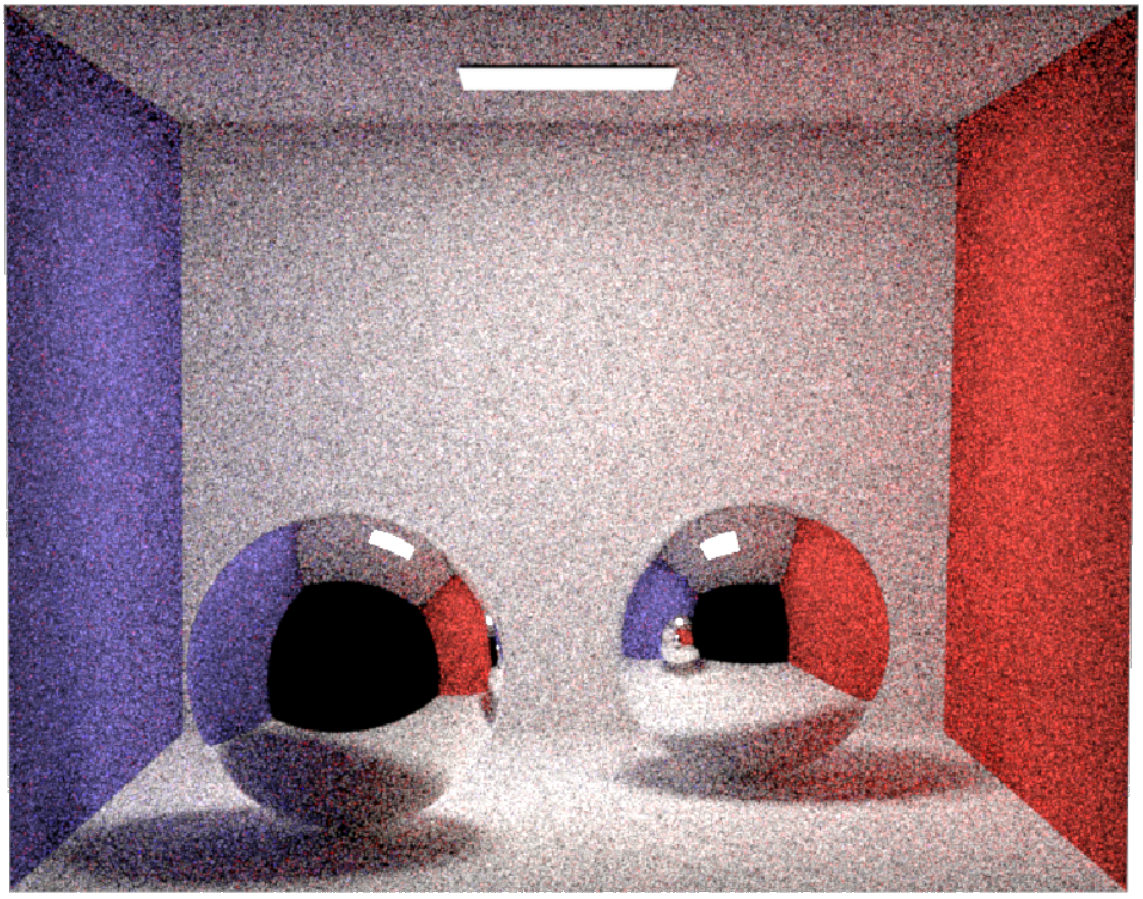

Figure 6: `pathtracer integrator`, 256 Samples, Mirror materials.

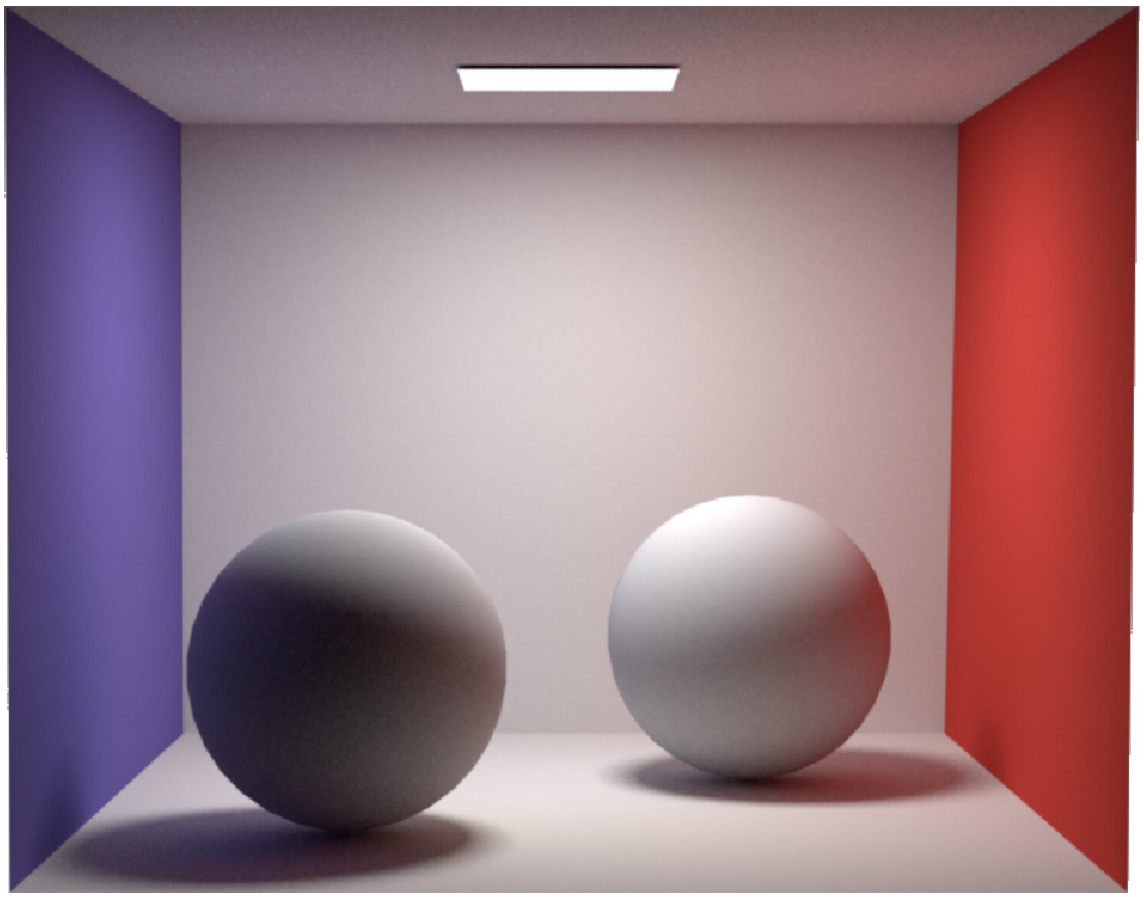

Figure 7: `pathtracer_nee integrator`, 256 Samples, Diffuse materials.

3 Direct + Indirect Illumination - `pathtracer_nee integrator`

The `pathtracer_nee integrator` works just like the `pathtracer integrator`, but at every intersection point we shoot two rays instead of one. The regular bounce ray we generate by importance sampling the material, and an additional one that we shoot towards a sample point in the chosen emitter, just like how light importance sampling worked in the `direct integrator`. We then weigh each ray's radiance contribution using MIS.

This technique is called next event estimation (NEE) because it explicitly estimates the contribution of the 'next event' (hitting a light source) right now, rather than waiting for it to happen naturally. In regular path tracing, a ray bounces randomly, and we only discover if it hit a light source at the next step of the recursion. This can make the renderer take a long time to converge if there are few light sources and they are small. With NEE, we explicitly shoot a shadow ray directly at the light source at the current step to compute direct illumination immediately.

As can be seen on Figure 7 and Figure 8, the render with mirror materials generates more noise. This is because perfect mirrors can't benefit from NEE, since the only direction the bounce ray can follow is the reflected direction of the previous ray, the one that intersected this mirror material. This means that the probability of generating a direction that hits the light is 0. So it doesn't matter if we sample the radiance of an emitter in the scene, because its MIS weight will always be 0.

However, fear not, with enough samples, as shown in Figure 9, the renderer still converges to a perfectly clean image by relying solely on the material bounces finding the light.

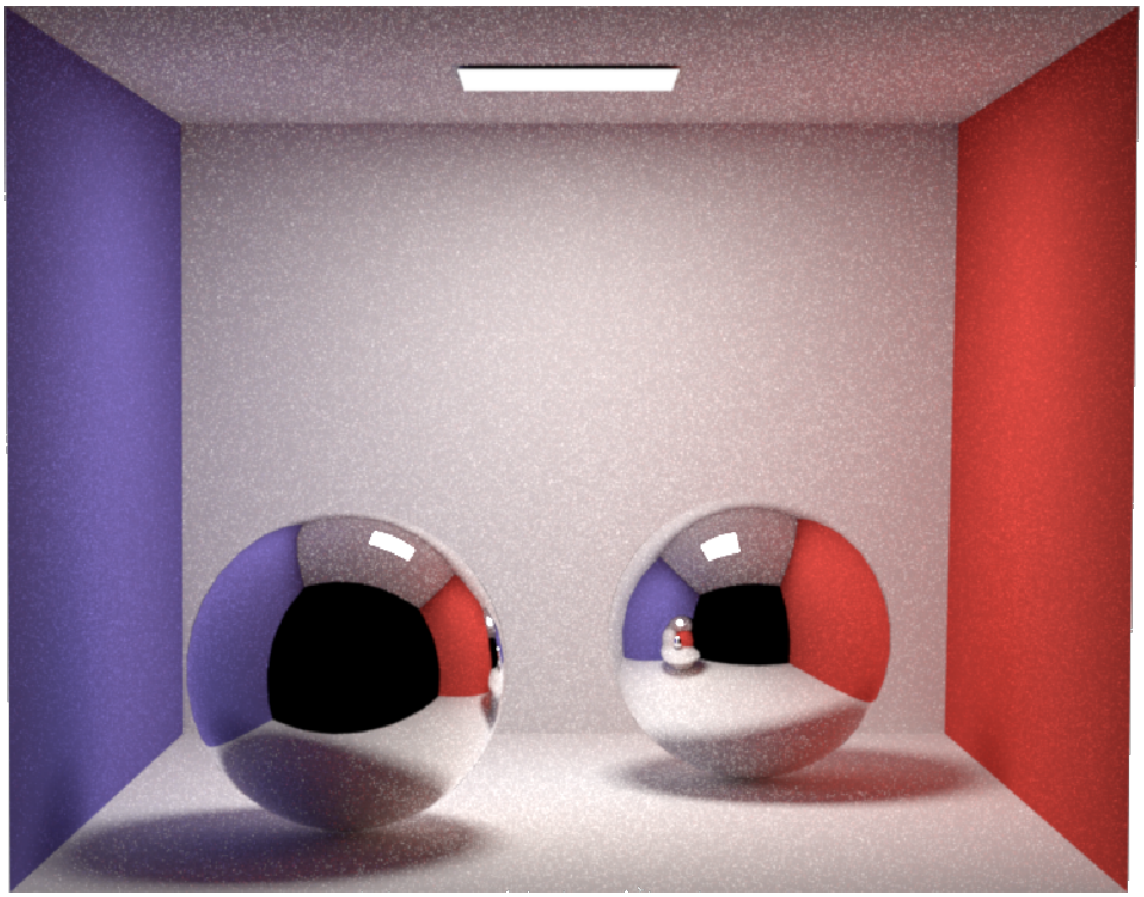

Figure 8: `pathtracer_nee integrator`, 256 Samples, Mirror materials.

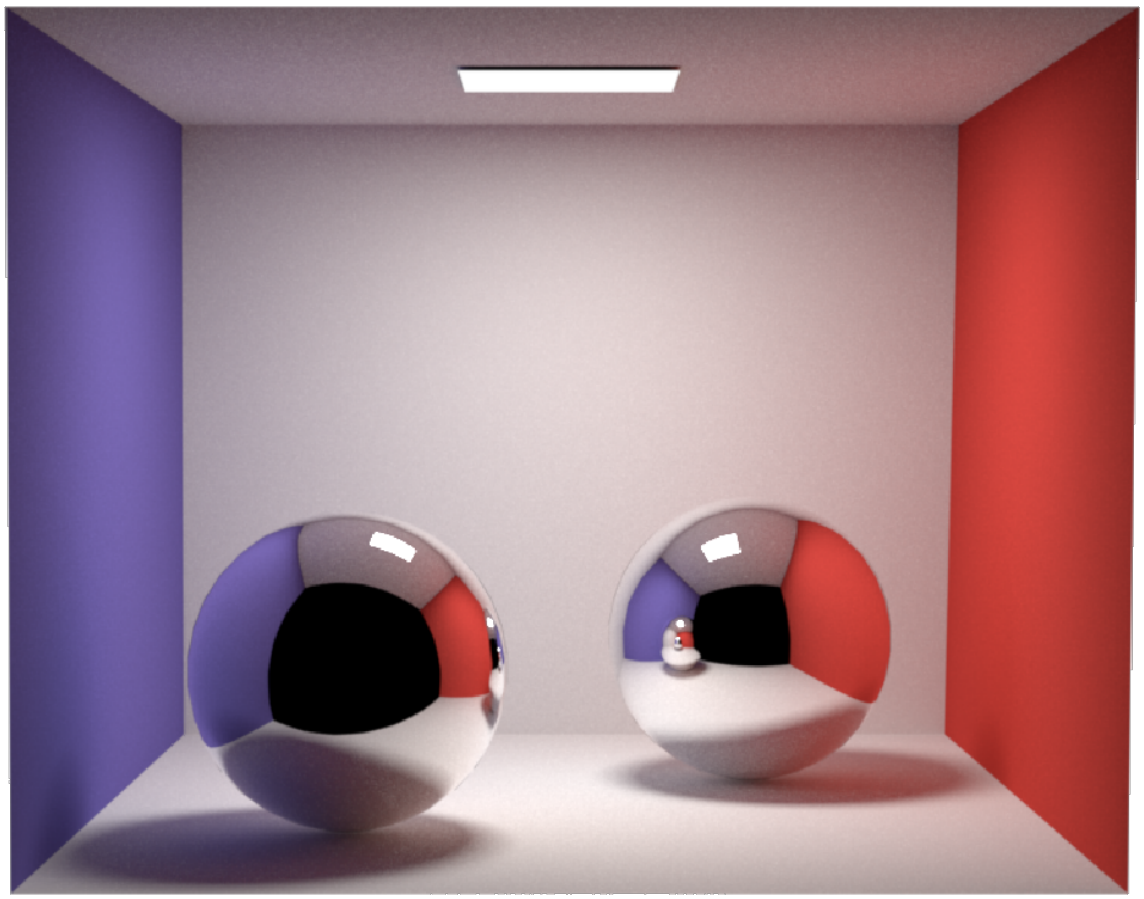

Figure 9: `pathtracer_nee integrator`, 4096 Samples, Mirror materials.

4 Materials - `microfacet bsdf`

In microfacet theory, an object's surface, known as the macrosurface, is modeled as a collection of many microscopic, perfectly smooth surfaces called microfacets. The fundamental idea of this theory is that instead of modeling each of these imperfections geometrically, which would be computationally infeasible, their aggregate behavior can be modeled statistically.

The central parameter in this statistical model is roughness α. This parameter determines the characteristics of the Normal Distribution Function D, which describes the orientation of the microfacet normals relative to the macrosurface normal, n. To be more specific, D is a function of $n \cdot h$, where h is the halfway vector between the outgoing direction and the incoming direction (Equation 2).

$$h = \frac{\omega_i + \omega_o}{\|\omega_i + \omega_o\|} \tag{2}$$

The halfway vector represents the direction a microfacet normal must have to reflect light from ω_i exactly toward ω_o.

A low roughness value implies a highly concentrated distribution of normals. The majority of microfacets are aligned with the main surface normal, producing sharp and well-defined specular reflections.

As roughness increases, the distribution of normals becomes wider and more dispersed, meaning the microfacets exhibit a large variation in their orientation. As a result, light is scattered across a much broader cone of directions, which diffuses the reflected light and produces blurry, soft reflections.

In addition to the distribution, roughness also influences the theory's second component, the Masking and Shadowing Function G. Since microfacets have varying heights and slopes, they can occlude one another from the viewpoint of the observer (masking) or from the light source (shadowing). A rougher surface, with greater variations in height, has a higher probability of this self-occlusion occurring. The G function statistically models this visibility.

The final component in the microfacet BSDF is the Fresnel reflectance F. When light hits the surface of an object, part is reflected out of the object into the scene, and part is refracted and travels deeper into the object. The amount of light that is reflected versus refracted is given by F. More specifically, F determines the percentage of total incident light that is reflected. Therefore, $1 - F$ is the percentage that is refracted.

And so, the specular microfacet BSDF in the **Cook-Torrance model** has the form shown in Equation 3.

$$f_s = \frac{F\ G\ D}{4\ (n \cdot \omega_i)(n \cdot \omega_o)} \tag{3}$$

On most physically-based materials to complete the bsdf this specular lobe is then weighted and summed up with a diffuse lobe, that is usually a simple Lambertian, like shown in Equation 4.

$$f = k_d\ f_d + k_s\ f_s \tag{4}$$

We can see this theory in action in Figure 10 and Figure 11.

- The yellow sphere in Figure 10 has a very, very low roughness material, and as a result, it's very glossy and reflective. We can see the reflection of the scene in it, almost like a mirror.
- The blue sphere in both Figure 10 and Figure 11 has a low roughness value, but not as low as the yellow sphere. As such, we can see the scene's light source in the pronounced specular reflection, but it's all much blurrier and we can barely see the surrounding objects reflected in it.
- The purple sphere in Figure 11 has a higher roughness material, and it looks like a regular diffuse material.

Now that we've explained the theory and shown how it looks, let's delve into the implementation in Nori's `microfacet` class that led to those materials. The class implements a physically-based layered material consisting of a Lambertian diffuse base and a GGX specular top layer. This is done through the three functions all of Nori's BSDFs must implement:

- `eval()`, which from the incoming ω_i and outgoing direction ω_o computes the value of the BSDF function.

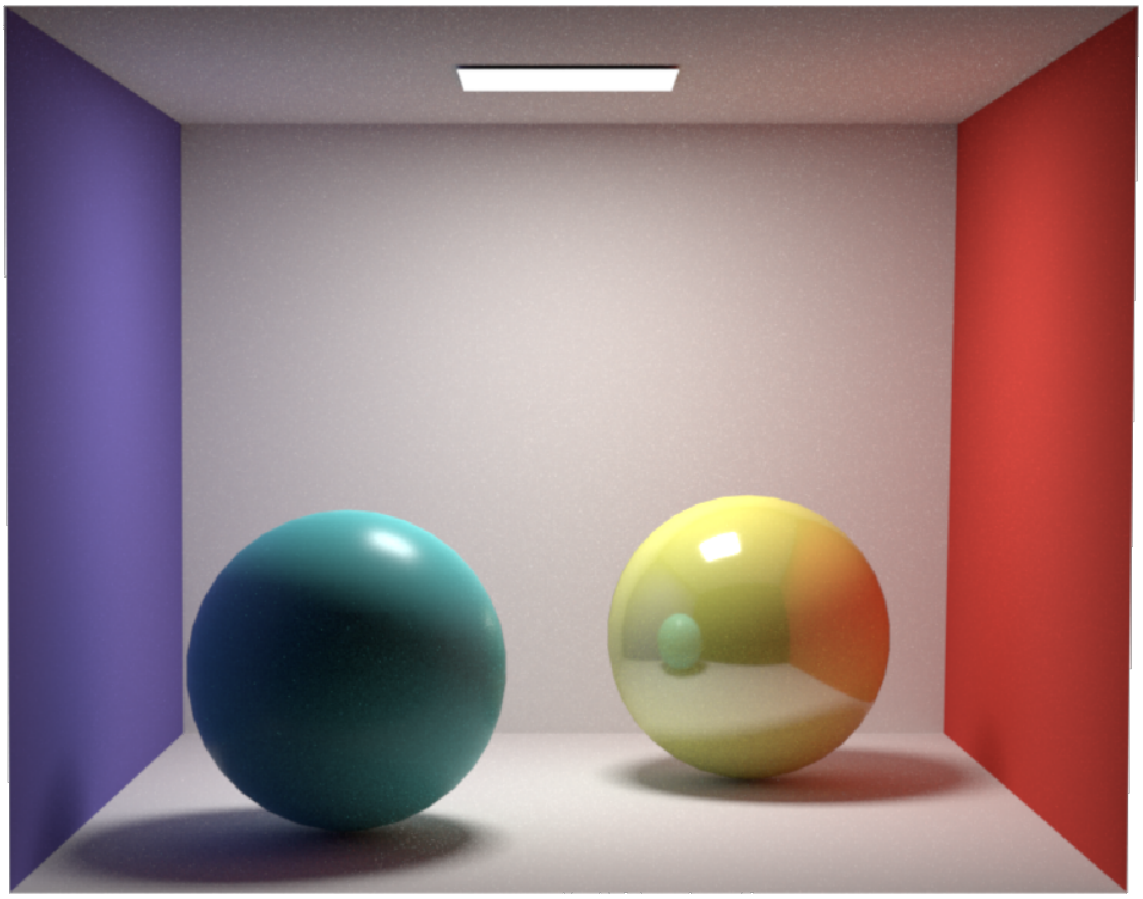

Figure 10: `pathtracer_nee integrator`, 512 Samples, Glossy and very glossy materials.

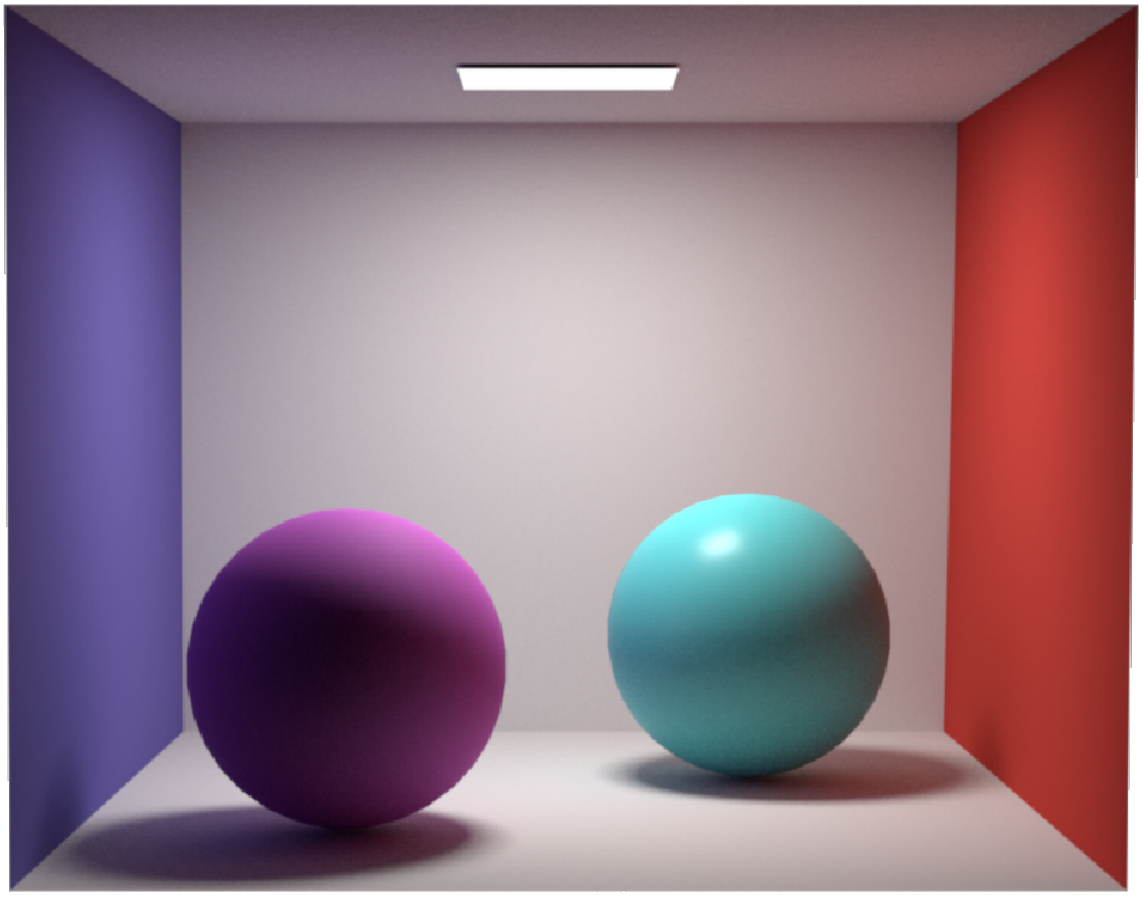

Figure 11: `pathtracer_nee integrator`, 512 Samples, Diffuse and glossy materials.

- `pdf()`, which from the incoming direction ω_i and outgoing direction ω_o computes the probability of generating that specific ω_o, given that the incoming direction was ω_i.
- `sample()`, which from the incoming direction ω_i generates the outgoing direction ω_o, and then returns the result of performing (`eval()` $(n \cdot \omega_i)$)/`pdf()` with those directions.

As we've covered before, the incoming direction ω_i usually denotes the direction from the light source to the shaded point, while the outgoing direction ω_o denotes the direction from the shaded point towards the previous shaded point in the recursion, or the camera. It's important to note that in Nori the incoming and outgoing directions are flipped. So, from now on we'll use Nori's conventions.

4.1 BSDF Evaluation - `eval()`

The `eval()` function computes the aggregate response of the material by summing the diffuse and specular components, weighted by k_d and k_s respectively. For simplicity's sake, k_d was chosen to be the material's albedo, while k_s is set to one minus the maximum component of k_d.

The specular component implements the Cook-Torrance model discussed earlier. For the **Normal Distribution Function** D, we implemented the function `D_GGX()`, which follows the isotropic version of the Trowbridge-Reitz distribution (better known as GGX), seen in Equation 5.

$$D(\alpha, n \cdot h) = \frac{\alpha^2}{\pi \left((\alpha^2 - 1)(n \cdot h)^2 + 1\right)^2} \tag{5}$$

For the geometric shadowing and masking, we implemented `V_GGX()`, which represents the **Visibility term** V. Because $V = \frac{G}{4(n \cdot \omega_i)(n \cdot \omega_o)}$, it saves us from having to divide by that denominator later.

We implemented the V used in the Frostbite engine (Equation 6). This implementation uses the Smith height-correlated masking-shadowing function. By using the height-correlated version, the model accurately accounts for the fact that a microfacet visible from the incoming direction is statistically more likely to be visible from the outgoing direction, which conserves energy better than older separable models.

$$V(\alpha, n \cdot \omega_i, n \cdot \omega_o) = \frac{1}{2\left((n \cdot \omega_i)\sqrt{\alpha^2 + (1 - \alpha^2)(n \cdot \omega_o)^2} + (n \cdot \omega_o)\sqrt{\alpha^2 + (1 - \alpha^2)(n \cdot \omega_i)^2}\right)} \tag{6}$$

For the **Fresnel** term, we used Nori's built in `fresnel` function that takes the IOR of the surface's exterior and the IOR of its interior.

Adding it all together, the BSDF formula ends up like shown in Equation 7.

$$f = k_d \frac{1}{\pi} + k_s \; F(h \cdot \omega_i, n_1, n_2) \; V(\alpha, n \cdot \omega_i, n \cdot \omega_o) \; D(\alpha, n \cdot h) \tag{7}$$

4.2 Probability - `pdf()`

The `pdf()` function returns the probability density of sampling a specific direction ω_o given ω_i, where the diffuse PDF follows a standard cosine-weighted distribution.

For the specular PDF, the implementation uses the **Visible Normal Distribution Function (VNDF)**. Rather than sampling the distribution of all normals D, we sample only those visible from the incoming direction ω_i. This has been shown to reduce variance because we're only sampling the microsurface normals actually seen by the camera.

Following Heitz's 2018 paper, to compute the distribution of visible normals D_v, we implemented the formula in equation Equation 8.

$$D_v = \frac{G1(\alpha, n \cdot \omega_i) \; max(0, h \cdot \omega_i) \; D(\alpha, n \cdot h)}{n \cdot \omega_i} \tag{8}$$

Here, $G1(\alpha, n \cdot \omega_i)$ comes from the separable masking-shadowing function. In the separable version, masking and shadowing are assumed to be independent events with no correlation, and as such the masking-shadowing function $G2$ can be computed like $G2 = G1(\alpha, n \cdot \omega_i) \cdot G1(\alpha, n \cdot \omega_o)$. This has been shown to not be the case, which is why we use the height-correlated $G2$ in our `eval()` function. Here, however, the $G1$ term is used to compute D_v. For $G1$, we employed the

formula in Equation 9, Smith's G1 term optimized for the GGX distribution, where s must be either ω_i or ω_o, depending on whether you want to model shadowing or masking.

$$G_1(\alpha, n \cdot s) = \frac{2}{1 + \sqrt{1 + \alpha^2 \frac{1-(n\cdot s)^2}{(n\cdot s)^2}}} \tag{9}$$

Once D_v has been computed, we can compute the probability of the sampled outgoing ray as seen in Equation 10.

$$p_s(\omega_o) = \frac{D_v}{4\ (h \cdot \omega_i)} \tag{10}$$

Lastly, the total PDF is a weighted linear combination of the diffuse and specular probabilities, which we'll touch on in the next section.

4.3 Sampling - `sample()`

The `sample()` function generates a new ray direction using importance sampling like we've been discussing for about 13 pages now.

The process begins by selecting which lobe to sample, diffuse or specular. The algorithm tosses a coin, assigning a probability proportional to $1 - k_s$ to the diffuse lobe and proportional to k_s to the specular lobe. This determines whether the current sample will follow the diffuse or specular distribution. These probabilities are the weights we use to compute the total PDF in the `pdf()` function, like we alluded to in the previous section.

If the diffuse lobe is chosen, the outgoing direction ω_o is sampled from a cosine-weighted hemisphere.

On the other hand, if the specular lobe is chosen, the implementation uses VNDF sampling, like we mentioned before. But, rather than using the traditional method, we obtain the microfacet normal sampling the VNDF using **spherical caps**, implemented in the `sampleGGX()` function following Dupuy's 2023 paper. This method is simpler and faster than Heitz's.

Once the microfacet normal h is obtained via this state-of-the-art method, the final outgoing direction ω_o is calculated by reflecting ω_i across h. The function finally returns the weighted contribution (`eval()` $(n \cdot \omega_o)$)/`pdf()`.